

অধ্যায় ১: এম্বেডেড সিস্টেম পরিচিতি

১.১ এম্বেডেড সিস্টেম কী?

ধরুন, আপনার বাসার (বা অফিসের) এসি (Air Conditioner) এর কথা। রিমোট দিয়ে আপনি টেম্পারেচার সেট করেন, ইনডোর ইউনিটের ফ্যানের স্পিড কম-বেশি করেন — আর এসি ঠিক সেই টেম্পারেচার ধরে রাখার জন্য বারবার কমপ্রেসার (Compressor) অন-অফ করে। আবার টেম্পারেচার সেম্পার দিয়ে রুমের তাপমাত্রা (Temperature) মেপে ডিসপ্লেতে দেখায়। এই পুরো কাজটি করে এসির ভেতরে বসানো একটি ছোট্ট কম্পিউটার। কিন্তু এই কম্পিউটারটি দিয়ে আপনি কখনো ফেসবুক চালাতে, গান শুনতে, বা গেম খেলতে পারবেন না। এটা শুধুমাত্র একটি কাজই জানে — কমপ্রেসার অন-অফ করার মাধ্যমে রুমের তাপমাত্রা নিয়ন্ত্রণে রাখা।

নোট: এখানে সরলভাবে বোঝানোর জন্য সাধারণ (Non-Inverter) এসির উদাহরণ দেওয়া হয়েছে। বর্তমানে বাজারে জনপ্রিয় Inverter AC গুলো কমপ্রেসার অন-অফ করার পরিবর্তে স্পিড কম-বেশি করে তাপমাত্রা নিয়ন্ত্রণ করে — যা আরও বেশি জটিল *এম্বেডেড সিস্টেম* এর উদাহরণ। এখানে মূলনীতি একই থাকে, শুধু নিয়ন্ত্রণ পদ্ধতি আরও বেশি সূক্ষ্ম হয়।

এটাই হলো **Embedded System** — একটি ছোট কম্পিউটার, যাকে কোনো একটি নির্দিষ্ট যন্ত্রের ভেতরে বসিয়ে দেওয়া হয়, আর তার কাজ একটাই: ওই একটিমাত্র নির্দিষ্ট কাজ (এক্ষেত্রে তাপমাত্রা নিয়ন্ত্রণ) বারবার, নির্ভুলভাবে এবং স্বয়ংক্রিয়ভাবে (Automatically) করে যাওয়া।

আপনার কম্পিউটার বা স্মার্টফোনের সাথে এর পার্থক্যটা লক্ষ্য করুন — কম্পিউটার হলো একটা “সবজান্তা”, যা দিয়ে ছবি এডিট, কোডিং, বা গেম খেলা — সবকিছুই করা যায়। কিন্তু Embedded System হলো একজন “স্পেশালিস্ট”, যে একটি কাজই করে, কিন্তু সেটা খুব ভালোভাবে করে।

এবার নিজের চারপাশে একটু তাকান — এই স্পেশালিস্ট কম্পিউটার কিন্তু সবখানে লুকিয়ে আছে:

- আপনার হাতের **স্মার্টওয়াচ** — শুধু আপনার হার্টবিট, স্টেপ কাউন্ট মাপার কাজ করে
- রাস্তার **ট্রাফিক লাইট** — শুধু লাল-হলুদ-সবুজ বাতি ঠিক সময়ে জ্বালা-নেভানোর কাজ করে
- ATM মেশিন — শুধু টাকা গোনা আর কার্ড রিড করার কাজ করে
- গাড়ির **এয়ারব্যাগ সিস্টেম** — শুধু দুর্ঘটনা বুঝে সাথে সাথে এয়ারব্যাগ খোলার কাজ করে

লক্ষ্য করুন, প্রত্যেক উদাহরণেই একটি কমন জিনিস আছে — প্রতিটি কম্পিউটার একটিমাত্র কাজে নিয়োজিত। এই “একটিমাত্র কাজে নিয়োজিত হওয়া” জিনিসটাই Embedded System-এর সবচেয়ে বড় বৈশিষ্ট্য।

এখন এর সাথে আরও দুটো ছোট বৈশিষ্ট্য যোগ করে নেওয়া যাক, যেগুলো পরের অধ্যায়গুলোতে বারবার কাজে লাগবে:

- লিমিটেড রিসোর্স:** আপনার ল্যাপটপে যেমন অনেক বেশি RAM আর Storage থাকে, Embedded System-এ কিন্তু তা থাকে না। যতটুকু তার কাজের জন্য দরকার, ঠিক ততটুকুই দেওয়া হয়। কারণ এখানে খরচ আর সাইজ দুটোই কম রাখতে হয়।
- দ্রুত রেসপন্স:** গাড়ির এয়ারব্যাগ সিস্টেমের কথা ভাবুন — দুর্ঘটনার পর যদি এয়ারব্যাগ খুলতে ১ সেকেন্ড দেরি হয়, তাহলে সেটার আর কোনো মূল্য নেই। এসি’র ক্ষেত্রেও ব্যাপারটা অনেকটা এমন — রুমের তাপমাত্রা বেড়ে গেলে সাথে সাথে কমপ্রেসার চালু হতে হয়, দেরি হলে ব্যবহারকারী স্বস্তি পাবেন না। তাই অনেক Embedded System-কে নির্দিষ্ট সময়ের মধ্যেই কাজ শেষ করতে হয় — একে বলে “রিয়েল-টাইম” আচরণ।

এই বইয়ে আমরা যে ESP32 নিয়ে কাজ করব, সেটাও Embedded System তৈরির মূল উপাদান — একে দিয়ে আমরা LED জ্বালানো থেকে শুরু করে নানা রকম নিবেদিত ও জটিল (Dedicated, Complex) কাজ সম্পাদন করাতে শিখব।

১.২ Microcontroller বনাম Microprocessor

এই দুটো টার্ম প্রায়ই গুলিয়ে ফেলা হয়, তাই শুরুতেই পার্থক্যটা বুঝে নেওয়া ভালো।

আপনার ল্যাপটপের ভেতরটা একটু কল্পনা করুন — CPU একটা আলাদা চিপ, RAM (মেমোরি) আরেকটা আলাদা স্টিক, আর স্টোরেজ (SSD/HDD) আরেকটা আলাদা কম্পোনেন্ট। এই আলাদা

আলাদা পার্টসগুলো মাদারবোর্ডে বসিয়ে, তার ও সার্কিটের মাধ্যমে একসাথে যুক্ত করার পরই একটা কার্যকর কম্পিউটার তৈরি হয়। এই ধরনের চিপ — যেখানে শুধু CPU থাকে, আর বাকি সবকিছু বাইরে থেকে জোগাড় করে যুক্ত করতে হয় — তাকে বলে **Microprocessor**।

এবার এমন একটা চিপ কল্পনা করুন, যার ভেতরেই CPU, মেমোরি (RAM), স্টোরেজ (Flash), আর ইনপুট/আউটপুট পিন — সবকিছু বিল্ট-ইন করা আছে। আলাদা করে কিছু জোগাড় করার দরকার নেই, একটামাত্র চিপ কিনলেই কাজ চলে যায়। এই ধরনের চিপকে বলে **Microcontroller (MCU)**।

বিষয়	Microprocessor	Microcontroller
গঠন	শুধু CPU থাকে, RAM ও I/O বাইরে থেকে যুক্ত করতে হয়	CPU, RAM, I/O — সব একই চিপে থাকে
উদাহরণ	Intel i5, i7 (আপনার ল্যাপটপের প্রসেসর)	ESP32, STM32, Arduino-এর ATmega
ব্যবহার	জেনারেল-পারপাস কম্পিউটিং (যেকোনো সফটওয়্যার চালানো যায়)	নির্দিষ্ট Embedded কাজ
দাম	তুলনামূলক বেশি	তুলনামূলক কম
পাওয়ার খরচ	বেশি	কম

ESP32 একটি **Microcontroller (MCU)** — মানে এর ভেতরেই CPU, মেমোরি, এবং অনেক Peripheral (যেমন GPIO, ADC, UART — এগুলো নিয়ে পরের অধ্যায়গুলোতে বিস্তারিত জানব) একই চিপে বসানো আছে। এই বইয়ে আমরা মূলত এই ধরনের MCU নিয়েই কাজ করব।

১.৩ ESP32 কী এবং কেন এত জনপ্রিয়?

ESP32 হলো চীনা কোম্পানি **Espressif Systems** তৈরি একটা low-cost, low-power System-on-Chip (SoC) সিরিজ, যেখানে Wi-Fi এবং Bluetooth বিল্ট-ইন আছে। ২০১৬ সালে রিলিজ হওয়ার পর থেকে এটা হবিস্ট, স্টুডেন্ট এবং প্রফেশনাল ডেভেলপার — সবার কাছেই সমানভাবে জনপ্রিয় হয়ে উঠেছে।

ESP32 কেন এত জনপ্রিয় তার কয়েকটা কারণ:

- দাম কম:** মাত্র কয়েকশ টাকায় (আনুমানিক ৪০০-৬০০ টাকা) পাওয়া যায়, অথচ ফিচার অনেক
- বিল্ট-ইন Wi-Fi ও Bluetooth:** IoT (Internet of Things) প্রজেক্টের জন্য এক্সট্রানাল মডিউল ছাড়াই কানেক্টিভিটি পাওয়া যায়
- Dual-Core প্রসেসর:** বেশিরভাগ ESP32 ভ্যারিয়েন্টে দুইটা কোর থাকে, যা একসাথে একাধিক কাজ (multitasking) চালাতে সাহায্য করে
- বিশাল কমিউনিটি:** সমস্যায় পড়লে সহজে সমাধান পাওয়া যায়, অসংখ্য লাইব্রেরি এবং উদাহরণ উপলব্ধ
- নমনীয়তা:** Arduino IDE বা ESP-IDF — দুই ধরনের ডেভেলপমেন্ট এনভায়রনমেন্টেই কাজ করা যায়

আমাদের বইয়ে ব্যবহৃত বোর্ড: ESP32 Dev Kit V1

এই বইয়ের সবগুলো উদাহরণ **ESP32 Dev Kit V1** বোর্ডে টেস্ট করা হয়েছে। এই বোর্ডের কিছু গুরুত্বপূর্ণ স্পেসিফিকেশন:

- চিপ:** ESP32-WROOM-32
- CPU:** Dual-core Xtensa LX6, 240 MHz পর্যন্ত ক্লক স্পিড
- RAM:** 520 KB SRAM
- ফ্ল্যাশ মেমোরি:** সাধারণত 4 MB (কিছু ভ্যারিয়েন্টে ভিন্ন হতে পারে)
- GPIO পিন:** প্রায় 25টা ব্যবহারযোগ্য GPIO
- কানেক্টিভিটি:** Wi-Fi (802.11 b/g/n), Bluetooth Classic ও BLE
- অন্যান্য পেরিফেরাল:** ADC, DAC, PWM, I2C, SPI, UART, Touch Sensor ইত্যাদি
- USB ইন্টারফেস:** প্রোগ্রামিং এবং সিরিয়াল কমিউনিকেশনের জন্য Micro-USB বা USB-C (বোর্ড ভেদে ভিন্ন হতে পারে) — সাধারণত CP2102 বা CH340 USB-to-Serial চিপ ব্যবহৃত হয়।

নোট: বাজারে বিভিন্ন ধরনের ESP32 Dev Kit V1 পাওয়া যায়। এদের মধ্যে সামান্য পার্থক্য থাকতে পারে, কিন্তু এই বইয়ের কোড সবগুলোতেই কাজ করবে।

১.৪ ESP-IDF কী?

ESP-IDF (Espressif IoT Development Framework) হলো Espressif কোম্পানির অফিসিয়াল সফটওয়্যার ডেভেলপমেন্ট ফ্রেমওয়ার্ক, যা ESP32 (এবং এর ফ্যামিলির অন্যান্য চিপ) প্রোগ্রাম করার জন্য ব্যবহৃত হয়। এটা তৈরি হয়েছে C এবং কিছুটা C++ এ, এবং এর ভিত্তি হলো **FreeRTOS** — একটি Real-Time Operating System।

সহজ ভাষায় বললে, ESP-IDF হলো টুলস, লাইব্রেরি এবং API-এর একটি সম্পূর্ণ প্যাকেজ, যা দিয়ে আপনি ESP32-এর প্রতিটি হার্ডওয়্যার ফিচার (Wi-Fi, Bluetooth, GPIO, Timer, ইত্যাদি) সরাসরি নিয়ন্ত্রণ করতে পারবেন।

ESP-IDF বনাম Arduino IDE — পার্থক্য কোথায়?

আপনি হয়তো ইতিমধ্যে জানেন যে ESP32 প্রোগ্রাম করার আরেকটা জনপ্রিয় উপায় হলো Arduino IDE। তাহলে ESP-IDF কেন শিখবেন?

বিষয়	Arduino Framework	ESP-IDF
শেখার কার্ড	সহজ, দ্রুত শুরু করা যায়	তুলনামূলক কঠিন, কিন্তু গভীর নিয়ন্ত্রণ দেয়
Abstraction	অনেক বেশি (হার্ডওয়্যার ডিটেইল লুকানো)	কম (হার্ডওয়্যার নিয়ে সরাসরি কাজ করা যায়)
RTOS নিয়ন্ত্রণ	সীমিত	সম্পূর্ণ FreeRTOS নিয়ন্ত্রণ (Task, Queue, Semaphore)
পারফরম্যান্স অপটিমাইজেশন	সীমিত সুযোগ	মেমোরি, পাওয়ার, স্পিড — সবকিছু ফাইন-টিউন করা যায়
প্রোডাকশন ব্যবহার	হবি/প্রোটোটাইপের জন্য উপযুক্ত	ইন্ডাস্ট্রি-গ্রেড প্রোডাক্টে ব্যবহৃত হয়
Configuration	সীমিত	menuconfig দিয়ে প্রতিটা মডিউল কাস্টমাইজ করা যায়

সহজ কথায়: Arduino দিয়ে আপনি দ্রুত একটি LED জ্বালাতে পারবেন, কিন্তু ESP-IDF দিয়ে আপনি বুঝবেন LED জ্বালানোর পেছনে ঠিক কী ঘটেছে, এবং একই সাথে প্রফেশনাল-লেভেল প্রজেক্ট (যেমন লো-পাওয়ার IoT ডিভাইস, মাল্টি-টাস্কিং সিস্টেম) তৈরি করতে পারবেন।

আপনি যদি চাকরির বাজারে বা প্রফেশনাল প্রজেক্টে ESP32 নিয়ে কাজ করতে চান, ESP-IDF শেখা প্রায় বাধ্যতামূলক — কারণ বেশিরভাগ কোম্পানি প্রোডাকশন কোডে Arduino ব্যবহার করে না।

১.৫ এই বইয়ে আপনি কী শিখবেন?

এই বইটা ধাপে ধাপে সাজানো হয়েছে, যাতে আপনি একদম বেসিক থেকে শুরু করে একটি সম্পূর্ণ IoT প্রজেক্টে বানানো পর্যন্ত পৌঁছাতে পারেন। বইয়ের শেষে আপনি যা করতে পারবেন:

- VSCode-এ ESP-IDF এনভায়রনমেন্ট সেটআপ ও ব্যবহার করা
- GPIO, Timer, Interrupt দিয়ে হার্ডওয়্যার নিয়ন্ত্রণ করা
- FreeRTOS দিয়ে মাল্টি-টাস্কিং প্রোগ্রাম লেখা
- I2C, SPI প্রোটোকল দিয়ে সেন্সর বা মডিউলের সাথে কমিউনিকেশন করা
- Wi-Fi দিয়ে ESP32-কে ইন্টারনেটে কানেক্ট করা এবং ডেটা পাঠানো
- একটি সম্পূর্ণ IoT প্রজেক্ট (সেন্সর → ক্লাউড) নিজে বানানো

১.৬ এই বই পড়ার আগে যা জানা থাকা ভালো

এই বইটা একদম “প্রোগ্রামিং না জানা” মানুষের জন্য নয়। শুরু করার আগে আপনার এই বিষয়গুলোতে বেসিক ধারণা থাকা প্রয়োজন:

- **C প্রোগ্রামিং:** Variable, Function, Loop, Pointer, Array — এই বিষয়গুলো সম্পর্কে ধারণা থাকতে হবে।
- **কম্পিউটার ব্যবহারে স্বচ্ছন্দ:** টার্মিনাল/কমান্ড লাইন ব্যবহার করতে পারা, ফাইল সিস্টেম বোঝা।
- **সাধারণ ইলেকট্রনিক্স জ্ঞান** (আবশ্যিক নয়, কিন্তু সহায়ক): ভোল্টেজ, কারেন্ট, রেজিস্টার, LED

এর মতো বেসিক কম্পোনেন্ট চেনা।

যদি এই বিষয়গুলোতে আপনি একদম নতুন হন, চিন্তার কিছু নেই — বইয়ের প্রতিটি প্রোগ্রাম উদাহরণসহ বিস্তারিতভাবে তুলে ধরা হবে, যাতে আপনি প্রতিটি বিষয় খুব সহজেই বুঝে নিতে পারেন।

১.৭ সারসংক্ষেপ

এই অধ্যায়ে আমরা জেনেছি:

- Embedded System কী এবং এর বৈশিষ্ট্য কী কী
- Microcontroller ও Microprocessor এর মধ্যে পার্থক্য
- ESP32 কেন এত জনপ্রিয়, এবং আমাদের ব্যবহৃত ESP32 Dev Kit V1 এর স্পেসিফিকেশন
- ESP-IDF কী এবং Arduino Framework এর সাথে এর পার্থক্য
- এই বইয়ে আমরা কী শিখব এবং কী প্রস্তুতি থাকা প্রয়োজন

পরবর্তী অধ্যায়ে আমরা VSCode-এ ESP-IDF ডেভেলপমেন্ট এনভায়রনমেন্ট সেটআপ করা শিখব, যাতে আপনি নিজের কম্পিউটারে প্রথম প্রোগ্রামটা রান করতে পারেন।

[পরবর্তী অধ্যায়: এনভায়রনমেন্ট সেটআপ — VSCode + ESP-IDF Extension]